杜教筛+狄利克雷卷积小结

作者
zhiyangfan shotacon

发布时间
2022-01-19 22:15

分类
个人记录

杜教筛+狄利克雷卷积小结

前话——狄利克雷卷积

定义

对于两个数论函数 f,g，我们定义它们的狄利克雷卷积 $f*g$ 为：

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

一些常见的数论函数：

- id，定义为 $\mathrm{id}(n)=n$，而 $\mathrm{id}_k(n)=n^k$。
- σ，定义为 $\sigma(n)=\sum_{d|n}d$，而 $\sigma_k(n)=\sum_{d|n}d^k$。
- ϵ，定义为 $\epsilon(n)=[n=1]$。
- φ，定义为 $\varphi(n)=\sum_{i=1}^{n}[\gcd(i,n)=1]$。
- μ，定义为，如果令 $n=\prod_{i=1}^{k}p_i^{c_i}$，其中 p_i 为质数，则

$$\mu(n)=\begin{cases}1 & n=1\\ (-1)^k & \prod_{i=1}^{k}c_i=1\\ 0 & \max\{c_i\}>1\end{cases}$$

- 1，定义为 $1(n)=1$。

积性函数

狄利克雷卷积的很多性质是围绕积性函数展开的，积性函数的定义是：

- 对于一个数论函数 f，如果 $f(1)=1$，且 $\forall\gcd(a,b)=1$，都有 $f(a)f(b)=f(ab)$，则称 f 为积性函数。特别地，如果 $\forall a,b$，都有 $f(a)f(b)=f(ab)$，则称 f 为完全积性函数。

可以发现，上文中的函数 $\mathrm{id},\epsilon,1$ 为完全积性函数，φ,μ,σ 是积性函数。

对于一个积性函数 f，它具有如下性质：

- $h(x)=f(x^p)$ 也是积性函数，如果注意到 $\gcd(a,b)=1$ 是 $\gcd(a^k,b^k)=1$ 的充分条件就能得到证明。
- $h(x)=f^p(x)$ 也是积性函数，可以根据乘积的幂等于幂的乘积证明。
- $h(x)=f(x)g(x)$ 也是积性函数，按照定义即可证明。
- $h(x)=(f*g)(x)$ 也是积性函数，依然可以直接套定义证明，只不过要稍微麻烦一点。

狄利克雷卷积与积性函数

可以通过狄利克雷卷积在积性函数之间建立起桥梁。

- $\mathrm{id}_k * 1 = \sigma_k$，其实就是：

$$(\mathrm{id}_k * 1)(n) = \sum_{d|n} d^k \cdot 1 = \sigma_k(n)$$

- $\varphi * 1 = \mathrm{id}$，证明如下：

  考虑令 $n = \prod_{i=1}^{k} p_i^{c_i}$，则现在的目的是要对 p^m，其中 p 是质数证明上式，剩下的可以通过积性函数的性质乘到一起。

  套定义有：

$$(\varphi * 1)(p^m) = \sum_{d|p^m} \varphi(d) = \sum_{d=1}^{m} \varphi(p^d) = \sum_{d=1}^{m} (p^d - p^{d-1}) = p^m = \mathrm{id}(p^m)$$

  得证。

- $\epsilon * f = f$，其中 f 是任意积性函数，其实就是：

$$(\epsilon * f)(n) = \sum_{d|n} \epsilon(d) f\left(\frac{n}{d}\right) = f(n)$$

  注意这其实反应了 ϵ 是狄利克雷卷积中的单位元。所以我们可以定义，如果 $f * g = \epsilon$，则称 f, g 在狄利克雷卷积中互逆。

- $1 * \mu = \epsilon$，证明如下： 依然考虑刚刚证明 $\varphi * 1 = \mathrm{id}$ 的套路：

$$(1 * \mu)(p^m) = \sum_{d|p^m} \mu(d) = \sum_{d=0}^{m} \mu(p^d) = [m = 0] = \epsilon(p^m)$$

  注意倒数第二步成立是因为 $\mu(1) = 1, \mu(p) = -1, \mu(p^m) = 0 (m > 1)$。

  注意这反映了 $1, \mu$ 是互逆的，所以这引出了莫比乌斯反演的一种形式：

$$f * 1 = g \Leftrightarrow f = g * \mu$$

正文——杜教筛与积性函数前缀和

众所周知，一般来说，随着毒瘤出题人的不断进步，我们在题目中常常会遇到如下式子：

$$\sum_{i=1}^{n} f(i)$$

其中 f 是一个积性函数。我们当然可以用欧拉筛做到 $\mathcal{O}(n)$ 的求解，但当 $n \leq 10^{12}$ 的时候，线性的时间复杂度就不够用了。这时候，亚线性的筛法，杜教筛就该上场了，它能做到在 $\mathcal{O}(n^{\frac{2}{3}})$ 的时间复杂度求解。

推式子

我们令 $S(n) = \sum_{i=1}^{n} f(i)$，则杜教筛的主要思想就是找到一个 $S(n)$ 关于 $S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$ 的递推式。

注意到对于任意一个数论函数 g，它一定满足：

$$\sum_{i=1}^{n} (f * g)(i) = \sum_{i=1}^{n} g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

根据定义有，$\sum_{i=1}^{n}(f*g)(i)=\sum_{i=1}^{n}\sum_{d|i}f(d)g(\frac{i}{d})$，注意到 $f(d)g(\frac{i}{d})$ 就是对所有 $d\le n$ 的做贡献，所以考虑交换求和顺序，并改为枚举 $d,\frac{i}{d}$（分别对应下式的 i,j）：

$$\sum_{i=1}^{n}\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}g(i)f(j)=\sum_{i=1}^{n}g(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}f(j)=\sum_{i=1}^{n}g(i)S\left(\left\lfloor\frac{n}{i}\right\rfloor\right)$$

得证。

所以我们就能得出杜教筛的主要式子：

$$g(1)S(n)=\sum_{i=1}^{n}(f*g)(i)-\sum_{i=2}^{n}g(i)S\left(\left\lfloor\frac{n}{i}\right\rfloor\right)$$

这个式子其实很简单，注意到根据上面证明的性质，右边第一项和第二项之差一个 $i=1$ 的情况，而这就等于左边。

而我们要做的主要任务就是寻找一个合适的 g，使 $f*g$ 性质足够好，能快速计算 $\sum_{i=1}^{n}(f*g)(i)$ 的值。

例题

> P4213 【模板】杜教筛（Sum）
>
> T 组数据，每组给出 n，分别求下面两个式子的值：
>
> $$\sum_{i=1}^{n}\varphi(i)\qquad\sum_{i=1}^{n}\mu(i)$$
>
> $(1\le T\le 10,1\le n\le 2^{31})$

板子题，我们分别来看这两个式子。

首先对于 φ，我们知道 $\varphi*1=\mathrm{id}$，而 $\sum_{i=1}^{n}\mathrm{id}(i)=\frac{1}{2}n(n+1)$ 非常好求。所以我们就选取 1 作为 g。

然后对于 μ，我们知道 $\mu*1=\epsilon$，而 $\sum_{i=1}^{n}\epsilon(i)=1$ 非常好求。所以我们就选取 1 作为 g。

分别选完之后套式子数论分块递归计算即可。直接算是 $\mathcal{O}(n^{\frac{4}{3}})$ 的，不过如果提前预处理出 $\le n^{\frac{2}{3}}$ 的答案，就能做到 $\mathcal{O}(n^{\frac{2}{3}})$ 了。

这里对于 φ 还有一种不需要单独找 g 的方法。考虑用 $\mathrm{id}*\mu=\varphi$ 的结论，原式即：

$$\sum_{i=1}^{n}\sum_{d|i}\frac{i}{d}\mu(d)=\sum_{d=1}^{n}\mu(d)\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}i=\sum_{d=1}^{n}\mu(d)\frac{(1+\lfloor\frac{n}{d}\rfloor)\lfloor\frac{n}{d}\rfloor}{2}$$

只要能快速求出 μ 的前缀和就可以数论分块做了。不过时间复杂度没有优化，依然是 $\mathcal{O}(n^{\frac{2}{3}})$。

```
#include <cstdio>
#include <unordered_map>
const int N = 2e6 + 10; typedef long long ll;
int mu[N], sumMu[N], p[N], vis[N], tp; std::unordered_map<ll, ll> mp;
inline void getMu()
{
    mu[1] = 1;
    for (int i = 2; i < N; ++i)
    {
```

```
        if (!vis[i]) p[++tp] = i, mu[i] = -1;
        for (int j = 1; j <= tp && (ll)i * p[j] < N; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { mu[i * p[j]] = 0; break; }
            mu[i * p[j]] = -mu[i];
        }
    }
    for (int i = 1; i < N; ++i) sumMu[i] = sumMu[i - 1] + mu[i];
}
ll Smu(ll n)
{
    if (n < N) return sumMu[n]; if (mp[n]) return mp[n];
    ll ret = 1;
    for (ll l = 2, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        ret -= Smu(n / l) * (r - l + 1);
    }
    return mp[n] = ret;
}
ll Sphi(ll n)
{
    ll ret = 0;
    for (ll l = 1, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        ret += (Smu(r) - Smu(l - 1)) * (1 + n / l) * (n / l) / 2;
    }
    return ret;
}
int main()
{
    int T; scanf("%d", &T); getMu();
    while (T--)
    {
        ll n; scanf("%lld", &n);
        printf("%lld %lld\n", Sphi(n), Smu(n));
    }
    return 0;
}
```

> P3768 简单的数学题
>
> 求：
>
> $$\sum_{i=1}^{n}\sum_{j=1}^{n} ij\gcd(i,j)$$
>
> 答案对一个给定的质数 p 取模。$(1 \le n \le 10^{10}, 5\times 10^8 \le p \le 1.1\times 10^9)$

需要一点莫比乌斯反演的板子题，常规莫反过程不再给出解释。

$$
\begin{aligned}
&\sum_{i=1}^{n}\sum_{j=1}^{n} ij\gcd(i,j)\\
=&\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{d=1}^{n} ij[\gcd(i,j)=d]\\
=&\sum_{d=1}^{n} d^3\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{d}\rfloor} ij[\gcd(i,j)=1]\\
=&\sum_{d=1}^{n} d^3\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{d}\rfloor} ij\sum_{p|i,p|j}\mu(p)\\
=&\sum_{d=1}^{n} d^3\sum_{p=1}^{\lfloor\frac{n}{d}\rfloor} p^2\mu(p)\sum_{i=1}^{\lfloor\frac{n}{dp}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{dp}\rfloor} ij\\
=&\sum_{t=1}^{n} t^2\sum_{d|t}\frac{t}{d}\mu(d)\left(\frac{(1+\lfloor\frac{n}{t}\rfloor)\lfloor\frac{n}{t}\rfloor}{2}\right)^2\\
=&\sum_{t=1}^{n} t^2\varphi(t)\left(\frac{(1+\lfloor\frac{n}{t}\rfloor)\lfloor\frac{n}{t}\rfloor}{2}\right)^2
\end{aligned}
$$

推完了，现在任务就很明白了，前面的 $\mathrm{id}_2\varphi$ 的前缀和用杜教筛算出来，后面的那一坨用数论分块。而用杜教筛我们就要找 g。

单纯的 φ 我们可以卷上一个 1 处理，而现在变成 $\mathrm{id}_2\varphi$，我们就考虑卷上一个 id_2。来看看如果令 $g=\mathrm{id}_2$ 会有什么结果：

$$
\begin{aligned}
\sum_{i=1}^{n}(f*g)(i)&=\sum_{i=1}^{n}((\mathrm{id}_2\varphi)*\mathrm{id}_2)(i)\\
&=\sum_{i=1}^{n}\sum_{d|i}(\mathrm{id}_2\varphi)(d)\mathrm{id}_2\left(\frac{i}{d}\right)\\
&=\sum_{i=1}^{n}\sum_{d|i} d^2\varphi(d)\left(\frac{i}{d}\right)^2\\
&=\sum_{i=1}^{n}\sum_{d|i} i^2\varphi(d)\\
&=\sum_{i=1}^{n} i^2\sum_{d|i}\varphi(d)=\sum_{i=1}^{n} i^3=\left(\frac{1}{2}n(n+1)\right)^2
\end{aligned}
$$

其中最后一步用了 $1*\varphi=\mathrm{id}$。发现这个结果还不错，所以选用 $g=\mathrm{id}_2$ 套杜教筛的公式即可。时间复杂度 $\mathcal{O}(n^{\frac{2}{3}})$。

```cpp
#include <cmath>
#include <cstdio>
#include <unordered_map>
const int N = 6e6 + 10; typedef long long ll;
int mod, inv6, inv4; ll n, pn; std::unordered_map<ll, ll> mp;
inline int ksm(int a, int b)
{
    int ret = 1;
    while (b)
    {
        if (b & 1) ret = (ll)ret * a % mod;
```

```
        a = (ll)a * a % mod; b >>= 1;
    }
    return ret;
}
int p[N], vis[N], phi[N], s[N], tp;
inline void getP()
{
    phi[1] = 1;
    for (int i = 2; i <= pn; ++i)
    {
        if (!vis[i]) p[++tp] = i, phi[i] = i - 1;
        for (int j = 1; j <= tp && (ll)p[j] * i <= pn; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { phi[i * p[j]] = (ll)phi[i] * p[j] % mod; break; }
            phi[i * p[j]] = (ll)phi[i] * phi[p[j]] % mod;
        }
    }
    for (int i = 1; i <= pn; ++i) s[i] = (s[i - 1] + (ll)i * i % mod * phi[i] % mod) % mod;
}
auto sum2 = [](ll x) { x %= mod; return x * (x + 1) % mod * (2 * x + 1) % mod * inv6 % mod; };
auto sum3 = [](ll x) { x %= mod; return x * x % mod * (x + 1) % mod * (x + 1) % mod * inv4 % mod;
};
int S(ll n)
{
    if (n <= pn) return s[n]; if (mp[n]) return mp[n];
    ll ret = sum3(n), pre = 1, cur;
    for (ll l = 2, r; l <= n; l = r + 1, pre = cur)
    {
        r = n / (n / l); cur = sum2(r);
        ret = (ret - S(n / l) * ((cur - pre + mod) % mod) % mod + mod) % mod;
    }
    return mp[n] = ret;
}
int work(ll n)
{
    int ret = 0, pre = 0, cur;
    for (ll l = 1, r; l <= n; l = r + 1, pre = cur)
    {
        r = n / (n / l); cur = S(r);
        ret = (ret + sum3(n / l) * ((cur - pre + mod) % mod) % mod) % mod;
    }
    return ret;
}
int main()
{
    scanf("%d%lld", &mod, &n); pn = pow(n, 2.0 / 3); getP();
    inv6 = ksm(6, mod - 2); inv4 = ksm(4, mod - 2); printf("%d\n", work(n)); return 0;
}
```

> P1587 [NOI2016] 循环之美
>
> 求出在 k 进制下有多少分数 $\frac{x}{y}$ 满足 $1 \le x \le n, 1 \le y \le n, x, y \in \mathbb{Z}$ 且对应的小数形式是纯循环的，一个小数是纯循环的当且仅当它能被表示成如下形式：
>
> $$a.\dot{c_1}c_2c_3\cdots c_{p-1}\dot{c_p}, a \in \mathbb{Z}, p \ge 1, c_i \in [0, k) \cap \mathbb{Z}$$
>
> 相同值的分数只算一次。($1 \le n, m \le 10^9, 2 \le k \le 2 \times 10^3$)

比较综合的一道题。

注意到一个分数 $\frac{x}{y}$ 在 k 进制下是纯循环的，当且仅当 $\gcd(y,k)=1$，而又因为相同值的分数只算一次，所以我们只统计最简分数就好，所以题目相当于在问：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1][\gcd(j,k)=1]$$

注意到 $[\gcd(j,k)=1]$ 这个跟常规的莫反不太一样，先不动它，先整掉前面的 $[\gcd(i,j)=1]$。

$$\begin{aligned}
&\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1][\gcd(j,k)=1]\\
=&\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d|i,d|j}\mu(p)[\gcd(j,k)=1]\\
=&\sum_{d=1}^{n}\mu(d)\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(jd,k)=1]\\
=&\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(jd,k)=1]
\end{aligned}$$

到这儿，如果继续用莫反展开那个 $[\gcd(jd,k)=1]$，能获得一个这样的式子：

$$\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\sum_{p=1}^{k}[p|k]\mu(p)\left\lfloor\frac{m}{\operatorname{lcm}(d,p)}\right\rfloor$$

因为 lcm 的限制，这个式子没办法继续优化了，只能 $\mathcal{O}(nk)$ 暴力枚举，获得 **64pts**。

看来用莫反展开行不通，注意到这个式子难化开的瓶颈在于 $[\gcd(jd,k)=1]$ 这个玩意，所以考虑把它拆开。（注意到 $\gcd(jd,k)=1$ 和 $\gcd(j,k)=1$ 且 $\gcd(d,k)=1$ 是互为充要条件）

$$\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor[\gcd(d,k)=1]\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(j,k)=1]$$

现在式子肉眼可见地被分为了两个部分，而我们的目标就是对这两个部分分别求前缀和。考虑设：

$$F(n)=\sum_{i=1}^{n}[\gcd(i,k)=1]\qquad S(n,k)=\sum_{d=1}^{n}\mu(d)[\gcd(d,k)=1]$$

现在就是要对这两个函数求出递归式。我们先考虑一眼看过去比较简单的 $F(n)$ 吧。观察到如果 $\gcd(i,k)=1$，则有 $\gcd(i+ak,k)=1,a\in\mathbb{N}$。这个可以通过辗转相除法的过程看出：

$$\gcd(i,k)=1\Leftrightarrow\gcd(k,i\bmod k)=1$$

所以先考虑 k 的剩余系中有多少与 k 互质的，再乘上 $\geq n$ 的数中能包含几个，最后算上剩下不完整的即可：

$$F(n)=\left\lfloor\frac{n}{k}\right\rfloor\varphi(k)+F(n\bmod k)$$

可以提前预处理出 $F(0\sim k)$ 的值，然后就能 $\mathcal{O}(1)$ 计算了，预处理的时间复杂度是 $\mathcal{O}(k\log k)$。

然后考虑 $S(n,k)$，化了半天也没什么思路，考虑先把 $[\gcd(d,k)=1]$ 用莫反展开：

$$
\begin{aligned}
S(n,k) &= \sum_{d=1}^{n} \mu(d)[\gcd(d,k)=1] \\
&= \sum_{d=1}^{n} \mu(d) \sum_{p|d,p|k} \mu(p) \\
&= \sum_{p=1}^{k} [p|k]\mu(p) \sum_{d=1}^{\lfloor \frac{n}{p} \rfloor} \mu(dp)
\end{aligned}
$$

这下难受了，$\mu(dp)$ 的求和是不连续的，不能用前缀和整出来。而如果想拆开，就必须满足 $\gcd(d,p)=1$，这也是无法保证的。等等，那如果 $\gcd(d,p)\neq 1$ 会怎么样？显然 dp 会出现重复的质因子，而这会导致 $\mu(dp)=0$！所以我们干脆直接套上一个 $\gcd(d,p)=1$ 的条件然后拆开就好，不会漏算东西。

$$
\begin{aligned}
&\sum_{p=1}^{k} [p|k]\mu(p) \sum_{d=1}^{\lfloor \frac{n}{p} \rfloor} \mu(dp) \\
=&\sum_{p=1}^{k} [p|k]\mu^2(p) \sum_{d=1}^{\lfloor \frac{n}{p} \rfloor} \mu(d)[\gcd(d,p)=1] \\
=&\sum_{p=1}^{k} [p|k]\mu^2(p) S\left(\left\lfloor \frac{n}{p} \right\rfloor, p\right)
\end{aligned}
$$

好了，这样强行拆开之后就得到了 $S(n,k)$ 的递归式了，而且总共只有 $\mathcal{O}(\sqrt{nk})$ 种状态，完全可以接受。但现在问题是，边界怎么办？显然有 $S(0,k)=0$，但是这还不够，$S(n,1)$ 我们也递归不下去。不过问题不大，因为当 $k=1$ 时，后面那个互质的条件就没了，即：

$$
S(n,1) = \sum_{i=1}^{n} \mu(d)
$$

n 很大，记忆化之后直接上杜教筛即可。

那这样这道题就结束了，时间复杂度 $\mathcal{O}(\sqrt{nk}+n^{\frac{2}{3}}+k\log k)$。用 `std::map` 记忆化的话会多一个 $\log$ 不过问题不大。

```cpp
#include <map>
#include <cmath>
#include <cstdio>
#include <algorithm>
const int N = 2e6 + 10; typedef long long ll; std::map<std::pair<int, int>, ll> mp;
int p[N], vis[N], mu[N], phi[N], sum[N], f[N], tp, pn, k;
inline void getP()
{
    mu[1] = phi[1] = vis[1] = 1;
    for (int i = 2; i <= pn; ++i)
    {
        if (!vis[i]) p[++tp] = i, mu[i] = -1, phi[i] = i - 1;
        for (int j = 1; j <= tp && (ll)p[j] * i <= pn; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { phi[i * p[j]] = phi[i] * p[j]; mu[i * p[j]] = 0; break; }
            phi[i * p[j]] = phi[i] * phi[p[j]]; mu[i * p[j]] = -mu[i];
        }
    }
    for (int i = 1; i <= pn; ++i) sum[i] = sum[i - 1] + mu[i];
}
```

```cpp
inline int F(int n) { return n / k * phi[k] + f[n % k]; }
ll S(int n, int k)
{
    if ((k == 1 && n <= pn) || (!n)) return sum[n];
    auto now = std::make_pair(n, k);
    if (mp[now]) return mp[now];
    if (k == 1)
    {
        ll ret = 1;
        for (int l = 2, r; l <= n; l = r + 1)
            r = n / (n / l), ret -= S(n / l, k) * (r - l + 1);
        return mp[now] = ret;
    }
    ll ret = 0;
    for (int i = 1; i * i <= k; ++i)
    {
        if (k % i) continue;
        if (mu[i]) ret += S(n / i, i);
        if (i * i != k && mu[k / i]) ret += S(n / (k / i), k / i);
    }
    return mp[now] = ret;
}
int main()
{
    int n, m; scanf("%d%d%d", &n, &m, &k); pn = N - 1; getP();
    for (int i = 1; i <= k; ++i) f[i] = f[i - 1] + (std::__gcd(i, k) == 1);
    ll ans = 0, cur, pre = 0;
    for (int l = 1, r; l <= std::min(n, m); l = r + 1, pre = cur)
    {
        r = std::min(n / (n / l), m / (m / l));
        cur = S(r, k); ans += (cur - pre) * (n / l) * F(m / l);
    }
    printf("%lld\n", ans); return 0;
}
```

作者：zhiyangfan　创建时间：2022-01-19 22:15:40

2　4　不推荐